

ÇALIŞILAN KONU BAŞLIK: **Proje Analizi, Veritabanı Tasarımı ve Backend Geliştirme**
(Node.js, Express, PostgreSQL, JWT)

YAPILAN İŞ

- Trello ve Jira analiz edilerek görev bazlı proje yönetimi incelendi.
- User story yöntemiyle kullanıcı ihtiyaçları belirlendi.
- ER diyagramı çıkarıldı (User, Project, ProjectMember, Column, Task).
- REST API endpoint listesi tasarlandı.
- Node.js + Express ile sunucu MVC mimaride kuruldu.
- Docker ile PostgreSQL 16 ayağa kaldırıldı; Prisma ORM + migration ile tablolar oluşturuldu.
- JWT ile kayıt/giriş yazıldı; şifreler bcrypt ile hash'lendi; Zod ile doğrulama eklendi.

KULLANILAN TEKNOLOJİLER

Node.js

Express

PostgreSQL

Docker

Prisma ORM

JWT

bcrypt

Zod

GÜNCEL / AKADEMİK YAKLAŞIM

Güvenli kimlik doğrulama.

Şifreler günümüz güvenlik standartlarında düz metin saklanmaz; tek yönlü hash (bcrypt) kullanılır. Oturum yönetiminde sunucuda durum tutmayan (stateless) JWT modern web mimarilerinde yaygındır.

KOD ÖRNEĞİ — KULLANICI KAYDI (ŞİFRE HASH + JWT)

```
auth.controller.js

export const register = async (req, res) => {
  const { name, email, password } = req.body;
  const hashed = await bcrypt.hash(password, 10); // şifreyi hash'le
  const user = await prisma.user.create({
    data: { name, email, password: hashed },
  });
  const token = signToken({ id: user.id, role: user.role }); // JWT üret
```

```
res.status(201).json({ user, token });  
};
```

EKRAN GÖRÜNTÜSÜ — GİRİŞ EKRANI

Giriş Yap

E-posta

Şifre

[Giriş Yap](#)

Hesabın yok mu? [Kayıt ol](#)

JWT ile güvenli giriş ekranı.

ÖĞRENİLENLER

Kazanımlar.

İstemci–sunucu mimarisi ve REST prensipleri pekiştirildi. Docker ile veritabanı kurulumu tek komuta indi; Prisma migration ile şema değişiklikleri sürüm kontrollü ilerledi.

ÇALIŞILAN KONU BAŞLIK: CRUD İşlemleri, React Arayüzü ve Sürükle-Bırak Kanban Panosu

YAPILAN İŞ

- Backend'de Proje, Sütun ve Görev için CRUD tamamlandı.
- Yeni projede otomatik To Do / Doing / Done sütunları açıldı.
- Yetkilendirme eklendi (üye olmayan erişemez → HTTP 403).
- Sürükle-bırak sıralaması transaction ile kalıcı kaydedildi.
- React + Vite ile frontend kuruldu; oturum Context ile yönetildi, JWT localStorage'da saklandı.
- Kanban panosu geliştirildi; @dnd-kit ile sürükle-bırak eklendi (optimistic update).
- Profil sayfası: ad güncelleme ve şifre değiştirme.

KULLANILAN TEKNOLOJİLER

React

Vite

React Router

Axios

@dnd-kit

Prisma \$transaction

GÜNCEL / AKADEMİK YAKLAŞIM

Bileşen tabanlı SPA ve optimistic UI.

Modern frontend'ler bileşen (component) tabanlıdır ve tek sayfa uygulaması (SPA) olarak çalışır. Kullanıcı deneyimi için iyimser güncelleme (optimistic UI) kullanılır: işlem sunucuya gitmeden arayüz anında güncellenir.

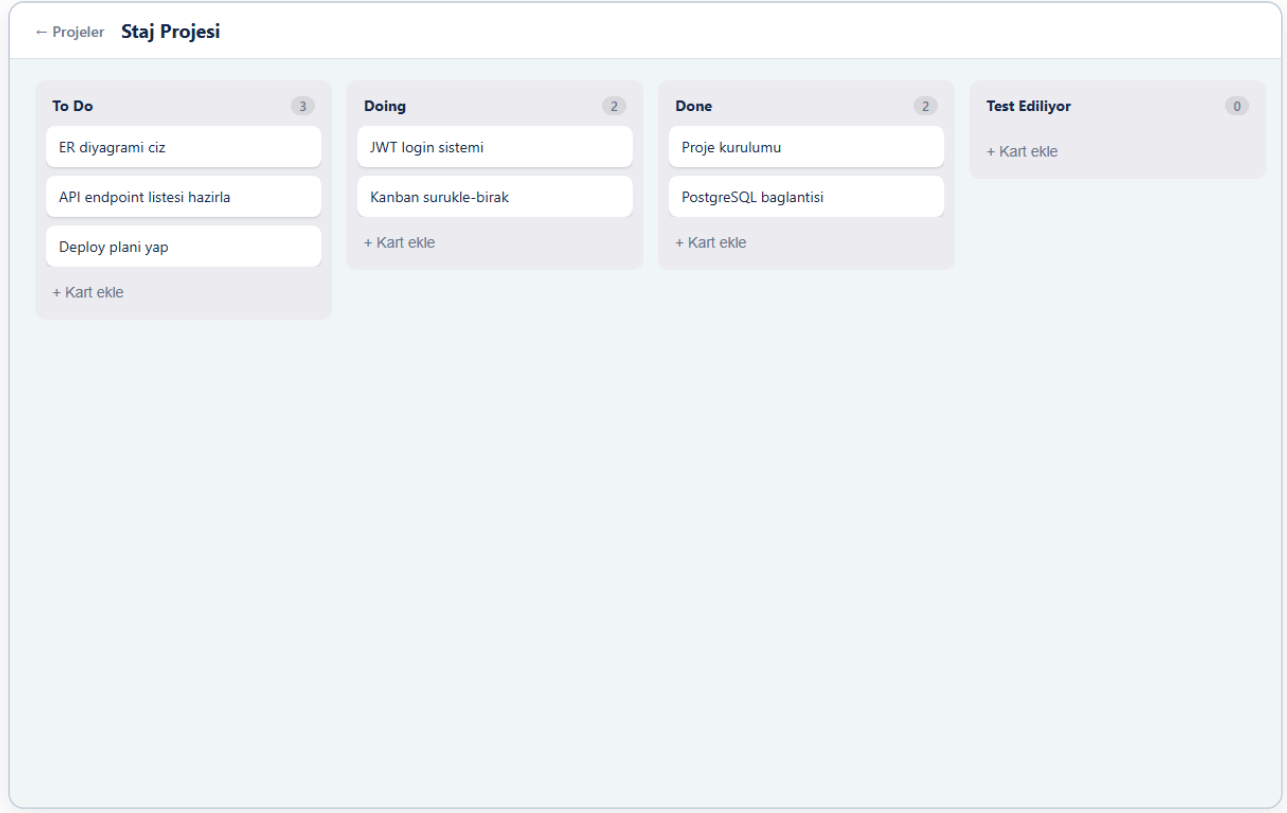
KOD ÖRNEĞİ — SİRALAMAYI TRANSACTION İLE KAYDETME

column.controller.js

```
await prisma.$transaction(  
  taskIds.map((taskId, index) =>  
    prisma.task.update({  
      where: { id: taskId },  
      data: { columnId, position: index }, // yeni sıra kalıcı kaydedilir  
    })  
  )  
)
```

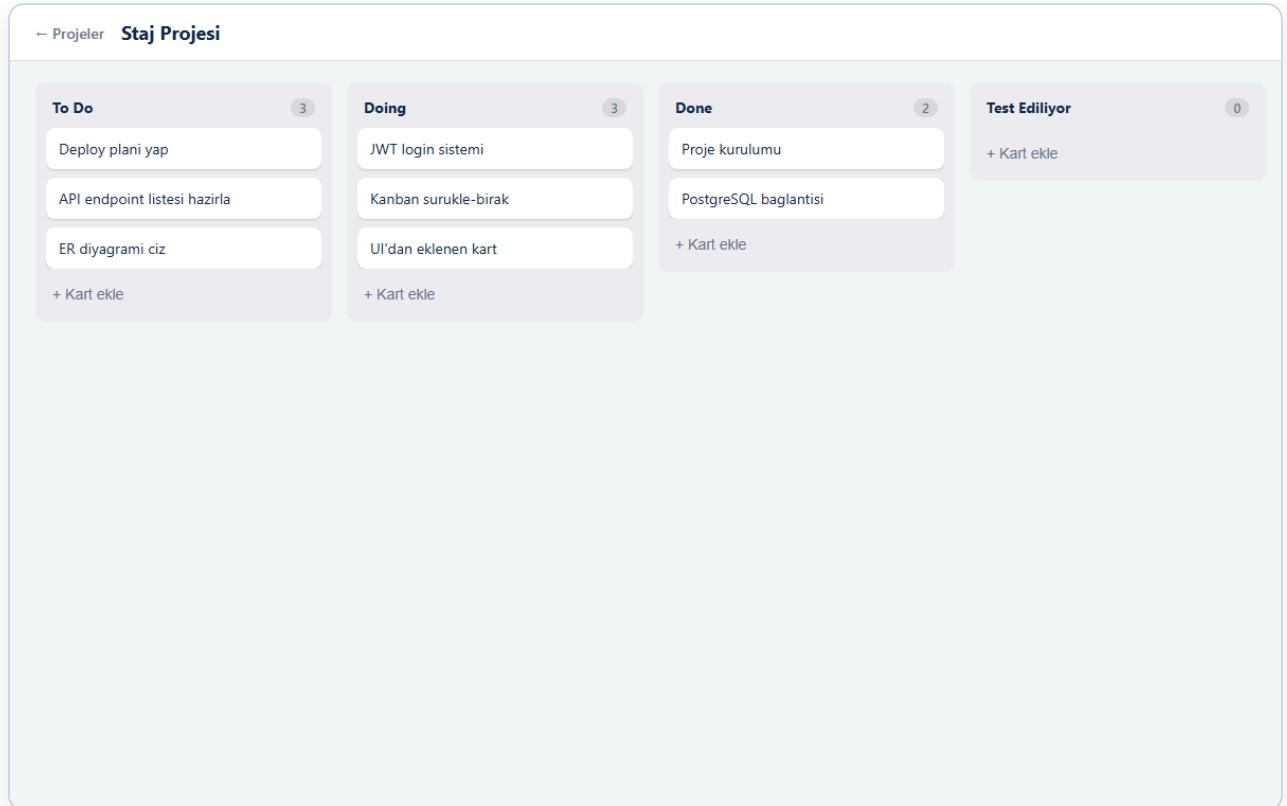
)
);

EKRAN GÖRÜNTÜSÜ — KANBAN PANOSU



Sütunlar ve kartlarla kanban panosu.

EKRAN GÖRÜNTÜSÜ — SÜRÜKLE-BIRAK SONRASI



Kart bir sütundan diğerine taşındıktan sonraki durum.

EKRAN GÖRÜNTÜSÜ — PROFİL VE ŞİFRE DEĞİŞTİRME

[← Projeler](#) **Profil**

Profil Bilgileri
E-posta
mustafa@test.com
Ad Soyad
Mustafa
Kaydet

Şifre Değiştir
Mevcut Şifre
Yeni Şifre
En az 6 karakter
Yeni Şifre (Tekrar)
Şifreyi Değiştir

Ad güncelleme ve şifre değiştirme ekranı.

ÖĞRENİLENLER

Kazanımlar.

React'te useState ve Context ile durum yönetimi kavrandı. Sürükle-bırakta sütunlar arası taşıma ve verinin backend'de tutarlı kalması (transaction) öğrenildi. Frontend ile backend'in CORS ve JWT üzerinden entegrasyonu görüldü.

ÇALIŞILAN KONU BAŞLIK: **Gelişmiş Görev Özellikleri (Öncelik, Son Tarih) ve Gerçek Zamanlı İşbirliği (WebSocket / Socket.io)**

YAPILAN İŞ

- Görev veri modeli genişletildi: öncelik (LOW / MEDIUM / HIGH) ve son tarih alanları eklendi, Prisma migration ile uygulandı.
- Karta tıklayınca açılan görev detay modalı yapıldı: başlık, açıklama, öncelik, son tarih ve atanmış kişi düzenlenebiliyor.
- Kanban kartlarına rozetler eklendi: öncelik rengi, son tarih ve atanmış kişinin baş harfi (avatar).
- Socket.io ile gerçek zamanlı işbirliği eklendi: bir kullanıcının yaptığı değişiklik, aynı projedeki diğer kullanıcılara sayfa yenilemeden anında yansıyor.
- Backend'de her görev/sütun değişikliğinde ilgili proje "odasına" WebSocket olayı yayınlandı (event-driven mimari).

KULLANILAN TEKNOLOJİLER

Socket.io

WebSocket

Prisma Migration

React useEffect

Zod enum

GÜNCEL / AKADEMİK YAKLAŞIM

Gerçek zamanlı iletişim (WebSocket) ve event-driven mimari.

HTTP'nin aksine WebSocket çift yönlü ve sürekli bir bağlantı kurar; sunucu istemciye kendiliğinden veri "itebilir" (server push). Olay tabanlı (event-driven) mimaride her değişiklik bir olay olarak yayınlanır. "Oda (room)" kavramıyla bildirim yalnızca ilgili projedeki kullanıcılara gider — bu, ölçeklenebilir bir yaklaşımdır.

KOD ÖRNEĞİ — SUNUCUDAN ANLIK BİLDİRİM (SOCKET.IO)

realtime.js — backend

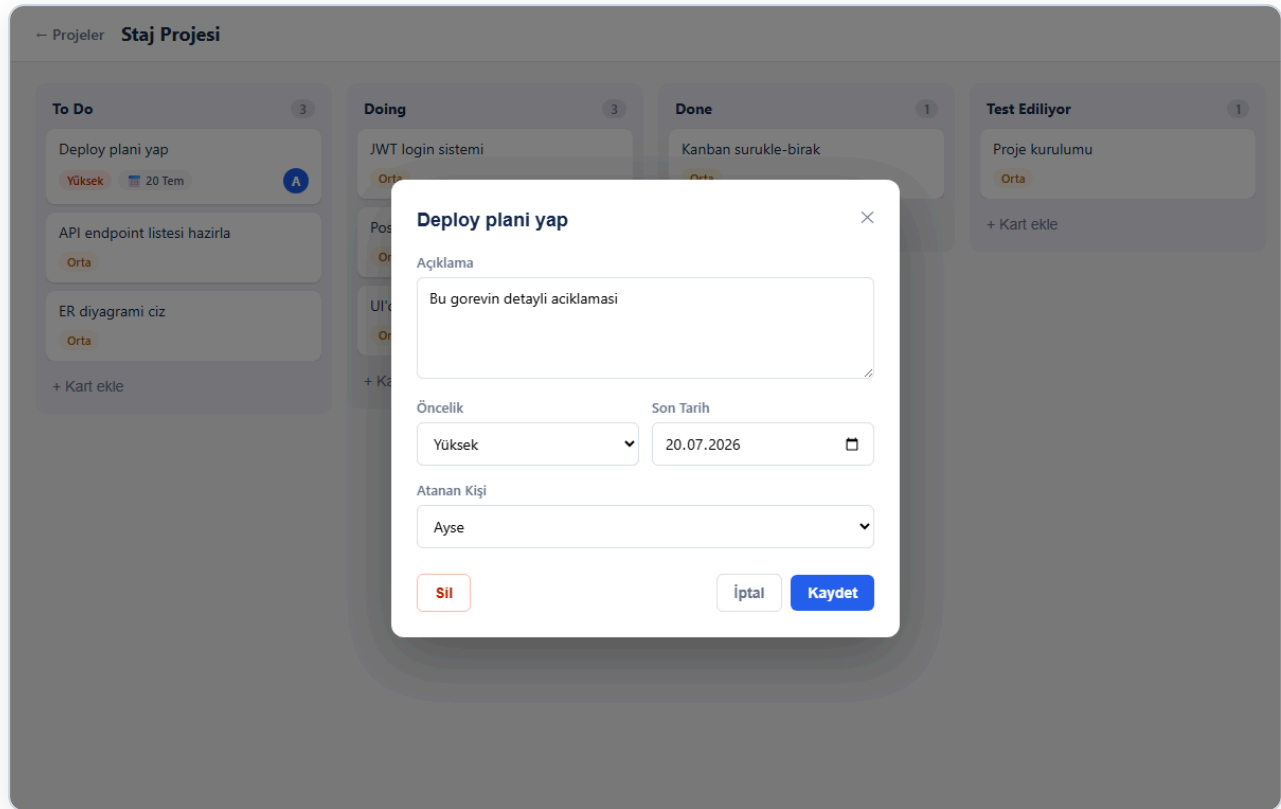
```
// Değişikliği yalnızca ilgili projenin odasına yayınla
io.to(`project:${projectId}`).emit("board:updated", { projectId });
```

KOD ÖRNEĞİ — İSTEMCİNİN OLAYI DİNLEMESİ (REACT)

Board.jsx — frontend

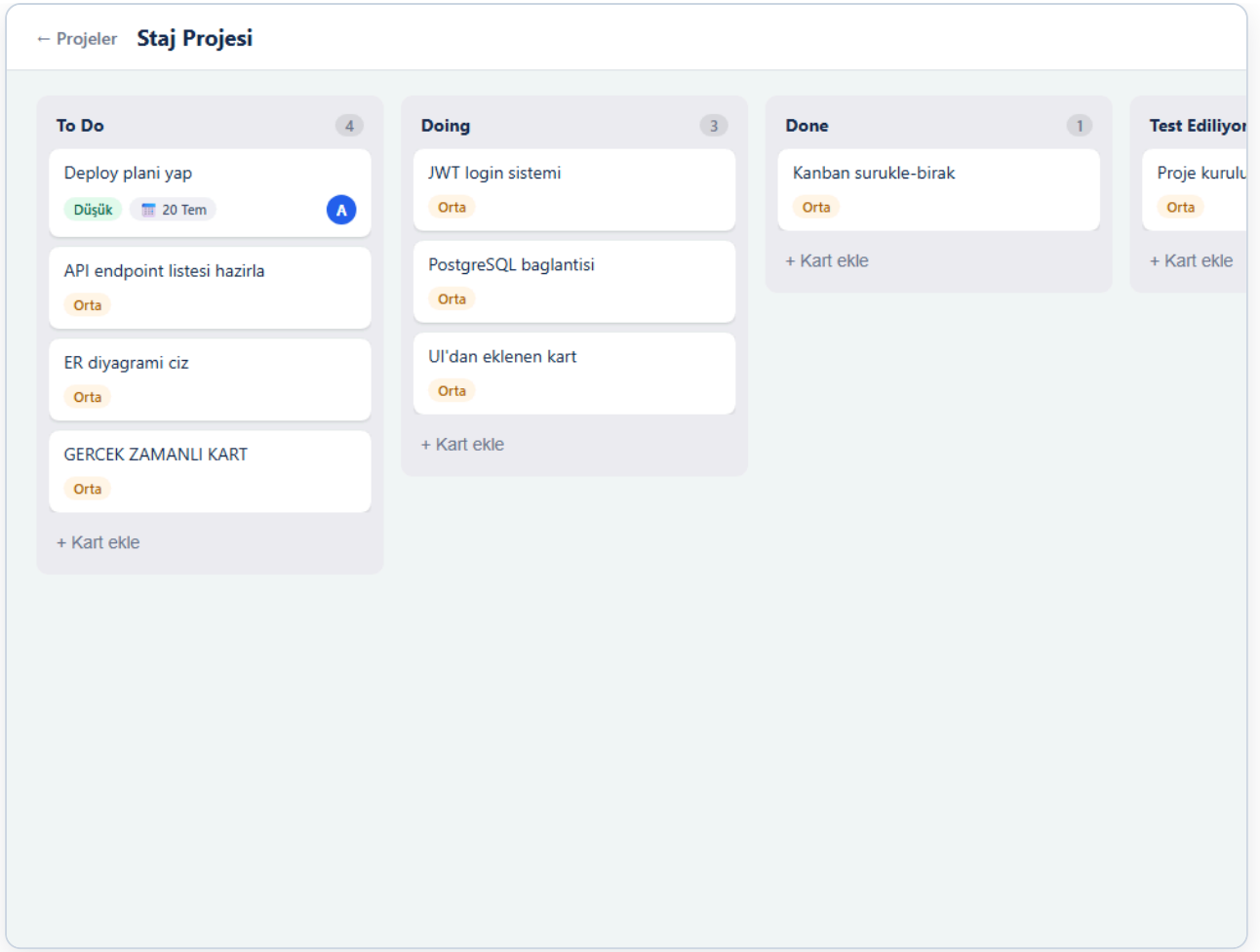
```
useEffect(() => {  
  socket.emit("join-project", id);           // projenin odasına katıl  
  socket.on("board:updated", () => load());  // değişiklik gelince tazele  
  return () => socket.off("board:updated");  
}, [id]);
```

EKRAN GÖRÜNTÜSÜ — GÖREV DETAY KARTI (MODAL)



Açıklama, öncelik, son tarih ve atanan kişi düzenleme; kartlarda renkli rozetler.

EKRAN GÖRÜNTÜSÜ — GERÇEK ZAMANLI GÜNCELLEME



İkinci kullanıcının ekranı: diğer kullanıcının eklediği kart, sayfa yenilenmeden otomatik belirdi.

ÖĞRENİLENLER

Kazanımlar.

WebSocket ile gerçek zamanlı iletişim mantığı öğrenildi. Veri modeline yeni alan eklerken migration süreci uygulandı. Event-driven mimari ve "oda (room)" kavramıyla yalnızca ilgili kullanıcılara bildirim gönderen ölçeklenebilir bir yapı kuruldu.